When Protections Silently Do Not Apply: A Benchmark of ASP.NET Core Security Misconfigurations and the Tools That Miss Them

Grant Totinov, Independent Researcher, Plovdiv, Bulgaria · ORCID 0009-0002-8014-409X

Technical report, version 1.0.0, 31 October 2026 · DOI 10.5281/zenodo.23822216

Abstract

ASP.NET Core lets developers turn on protections such as antiforgery validation, rate limiting and CORS with a few lines of configuration. When those lines are ordered or combined incorrectly, the protection can stop applying while the application starts normally, serves requests and logs nothing. This report presents a benchmark of such silent misconfigurations: pairs of minimal ASP.NET Core applications that differ only in one misconfiguration, each verified at runtime on .NET 8 and .NET 10.

The first release covers four misconfigurations: a form-field HTTP method override that bypasses antiforgery validation, a rate limiter registered before routing, an MVC controller without antiforgery validation, and a CORS policy that grants any origin credentialed access. Three static analysis tools were evaluated: the .NET SDK's built-in analyzers with every rule enabled, Semgrep with its public C# rules, and CodeQL with its C# security-extended suite.

None of the tools detected any of the four misconfigurations. All three ship a rule for the MVC antiforgery case, yet each misses it. Two of them, by design, stay silent in projects that never use an antiforgery attribute, so the application with no protection at all gets no warning. The third does not match a common attribute syntax and also flags the recommended fix. The benchmark, scripts and results are openly available.

1 Introduction

In ASP.NET Core, much of an application's security is configured in code. Developers register middleware in a specific order, attach policies to endpoints and add filters to controllers. The framework provides the protections, but whether they apply depends on how the application wires them together.

Some wiring mistakes fail loudly: the application throws at startup or a request fails. Others fail silently. The protection is configured, the code looks correct at a glance, and the application runs, but for some requests the protection never executes. Nothing in the logs says so.

The author encountered one such case while contributing to ASP.NET Core. When form-field HTTP method override runs before antiforgery validation, a cross-site form POST can be turned into a DELETE that antiforgery never checks. That interaction is now documented in the framework's API reference and in the ASP.NET Core documentation, following the author's contributions [6, 14, 15]. Documentation, however, only helps developers who read it. This raises the question whether the tools developers run on their code catch such cases.

This report addresses three research questions:

1. **RQ1.** Do these misconfigurations actually disable the protection at runtime on current .NET versions, and does the framework warn about them?
2. **RQ2.** Do widely used static analysis tools detect them?
3. **RQ3.** When a tool has a rule for a misconfiguration, why does it miss it?

The contributions are:

- A benchmark of four misconfiguration pairs, each a vulnerable and a fixed ASP.NET Core application, verified over HTTP on .NET 8 and .NET 10 by an automated runner that also runs in continuous integration.
- An evaluation of three static analysis tools: the built-in .NET analyzers, Semgrep and CodeQL. Each detects 0 of the 4 misconfigurations.
- A root-cause analysis of why the three tools' CSRF rules miss the MVC case, with minimal reproductions.
- The benchmark code, runner and raw results, published together with this report [13].

2 Background

Middleware pipeline. An ASP.NET Core application processes each request through middleware components in the order they are registered. `UseRouting()` selects the endpoint that will handle the request. Middleware registered after it can read that endpoint's metadata, such as a rate limiting policy or an antiforgery requirement; middleware registered before it cannot [16].

Antiforgery. ASP.NET Core defends against cross-site request forgery (CSRF, CWE-352) with synchronizer tokens. Since .NET 8, `UseAntiforgery()` adds middleware that validates tokens for endpoints carrying antiforgery metadata, which includes minimal API endpoints that bind form data. It validates only POST, PUT and PATCH requests [6]. MVC controllers are protected differently: by the `[ValidateAntiForgeryToken]` attribute on an action or controller, or by a global `AutoValidateAntiforgeryToken` filter, which the documentation recommends for non-API applications [6]. Razor Pages validate tokens by default.

HTTP method override. `UseHttpMethodOverride()` lets clients limited to GET and POST, such as HTML forms, request other methods. With `FormFieldName` set, it rewrites a POST into the method named in that form field before later middleware runs [17].

Rate limiting. The rate limiting middleware applies global limiters and named policies. Policies attached to endpoints with `RequireRateLimiting()` or `[EnableRateLimiting]` require `UseRateLimiter()` to be called after `UseRouting()` [7].

CORS. A CORS policy decides which origins may read responses in the browser. With `AllowCredentials()`, the browser also sends cookies. ASP.NET Core refuses `AllowAnyOrigin()` together with `AllowCredentials()`, and the documentation names `SetIsOriginAllowed(_ => true)` with `AllowCredentials()` as an equally insecure way around that check [8].

3 Benchmark design

Selection criteria. A misconfiguration enters the benchmark only if it meets four conditions:

1. It lives in application code and uses only built-in ASP.NET Core APIs.
2. The protection silently does not apply: the application starts, throws no exception and logs no warning.
3. Official Microsoft documentation describes the configuration as incorrect or insecure.
4. The effect is observable over HTTP, so it can be verified without a browser or a debugger.

Pairs. Each rule is a pair of minimal applications, `Vulnerable` and `Fixed`, that differ only in the lines that make up the misconfiguration. The fixed variant follows the remedy that the official documentation recommends. The applications contain only what the rule needs: a `Program.cs`, and a controller where MVC is involved. None uses authentication or a database. Because everything else is identical, any difference in a tool's output between the two variants can be attributed to the misconfiguration.

Runtime verification. A runner, a C# console application that uses only the base class library, builds every variant for `net8.0` and `net10.0`, starts it on a free loopback port in the `Production` environment and sends three kinds of request:

- an **attack** request that the protection should stop;
- a **legitimate** request that must keep working, so that a fix which simply breaks the feature fails the check;
- where useful, a **control** request showing that the protection is active for ordinary requests.

A vulnerable variant passes when the attack succeeds and the legitimate request works. A fixed variant passes when the attack is blocked and the legitimate request still works. The runner stores each application's log, which was inspected for warnings. It ran locally on Ubuntu 24.04 with ASP.NET Core runtimes 8.0.33 and 10.0.14, and it runs in GitHub Actions on every change to the repository.

4 The four misconfigurations

All four misconfigurations disable their protection at runtime on both .NET 8 and .NET 10, and none of them produces a warning in the application log (RQ1). Table 1 summarizes the verified behavior.

R01: form-field method override before antiforgery (CWE-352). The pipeline calls `UseHttpMethodOverride` with `FormFieldName = "_method"`, then `UseRouting()` and `UseAntiforgery()`. The target is a DELETE endpoint that binds form data, so it carries antiforgery metadata. A form POST with `_method=DELETE` and no token is routed to that endpoint, and antiforgery skips it because DELETE is not a validated method. A plain POST without a token is still rejected, which shows antiforgery is otherwise active. The fixed variant keeps the pipeline and validates the token in the endpoint with `IAntiforgery.IsRequestValidAsync`, as the documentation recommends when the override must run first [6].

R02: rate limiter before routing (CWE-307). The application registers `UseRateLimiter()` before `UseRouting()`. A login endpoint carries a fixed-window policy of 3 requests per minute with no queue. Because no endpoint has been selected when the limiter runs, the policy never applies. The fixed variant swaps the two calls [7].

R03: MVC controller without antiforgery validation (CWE-352). The application calls `AddControllersWithViews()` and exposes a state-changing `[HttpPost("email")]` action. Neither the action, the controller nor a global filter requests token validation, so the action accepts requests without a token. The fixed variant adds `AutoValidateAntiforgeryTokenAttribute` as a global filter [6].

R07: CORS with any origin and credentials (CWE-942). The default policy combines `SetIsOriginAllowed(_ => true)` with `AllowCredentials()`. The middleware echoes every request's origin in `Access-Control-Allow-Origin` and adds `Access-Control-Allow-Credentials: true`. The fixed variant lists the trusted origin with `WithOrigins` [8]. For comparison, the direct combination `AllowAnyOrigin()` with `AllowCredentials()` makes the application fail at startup with an `InvalidOperationException` on .NET 10.0.14; the predicate form passes that check.

Table 1. Runtime behavior. Results were identical on .NET 8.0.33 and .NET 10.0.14.

Rule	Weakness	Attack request	Vulnerable	Fixed
R01	CWE-352	POST with `_method=DELETE`, no token	200, item deleted	400
R02	CWE-307	10 login attempts within a minute (limit 3)	10 × 200	3 × 200, 7 × 429
R03	CWE-352	POST to the MVC action, no token	200, email changed	400
R07	CWE-942	GET from `Origin: https://evil.example`	origin echoed, credentials allowed	no CORS headers

In all 16 runs (4 rules × 2 variants × 2 frameworks), the legitimate request succeeded in both variants: a DELETE or POST with a valid token (R01, R03), the first login attempt (R02), and a GET from the trusted origin, which received the CORS headers (R07).

5 Tool evaluation

Three freely available static analysis tools were evaluated, each in its broadest documented security configuration (Table 2). All runs happened in GitHub Actions on Ubuntu 24.04 on 30 October 2026; the built-in analyzers were also run locally with the same result.

Table 2. Tools and configuration.

Tool	Version	Configuration	Input
Built-in .NET analyzers	.NET SDK 10.0.403 (CI), 10.0.114 (local)	`AnalysisLevel=latest-all`, which enables every CA rule including the security rules that are off by default; ASP.NET Core analyzers from the .NET 8 and .NET 10 targeting packs	A full build of each variant for `net8.0` and for `net10.0`
Semgrep	1.183.0	All 52 C# rule files from the public semgrep-rules repository, commit 16b9e6d	The source folder of each variant
CodeQL	2.27.3	C# `security-extended` suite, query pack csharp-queries 1.10.2	A database built from the `net10.0` build of each variant

Scoring. Because the two variants of a rule differ only in the misconfiguration, a tool **detects** a rule if it reports a finding on the vulnerable variant whose rule identifier it does not also report on the fixed variant. Every finding on the fixed variant is recorded separately: it is either a false positive for the misconfiguration or an unrelated finding on code the variants share. For the built-in analyzers, only the security category of the CA rules and all ASP.NET Core analyzers are scored. Style and design diagnostics are kept in the raw data. Semgrep and CodeQL write SARIF, which the runner scores with the same rule. All findings were also inspected manually.

6 Results

No tool detected any of the four misconfigurations (RQ2). Table 3 gives the result per rule. The built-in analyzers were run on both frameworks and detected 0 of 8 rule and framework pairs.

Table 3. Detection per rule and tool.

Rule	Built-in .NET analyzers	Semgrep	CodeQL
R01 method override before antiforgery	not detected	not detected	not detected
R02 rate limiter before routing	not detected	not detected	not detected
R03 MVC without antiforgery	not detected	not detected	not detected
R07 CORS, any origin with credentials	not detected	not detected	not detected
Detected	**0 of 4**	**0 of 4**	**0 of 4**

The tools did run. The built-in analyzers reported style and design rules on the same projects (CA1050, CA1515 and CA2007), and none in the security category. Semgrep's only finding, `missing-or-broken-authorization`, appeared on both R03 variants and concerns the absence of authentication, not CSRF. CodeQL reported nothing on any variant, so no tool produced a false positive on a fixed variant.

For R01, R02 and R07, no tool reported anything related to the misconfiguration. Semgrep's 52 C# rules and CodeQL's C# query pack contain no rule about CORS policies, rate limiter placement or method override. R03 is different: all three tools ship a rule for it, which Section 7 examines.

7 Why the CSRF rules miss R03

The three tools miss R03 for two different reasons (RQ3). CA5391 and CodeQL suppress the warning by design when a project uses no antiforgery attribute at all. Semgrep's rule does not match the attribute syntax R03 uses.

7.1 CA5391 and CodeQL: silent unless protection is already in use

CA5391, "Use antiforgery tokens in ASP.NET Core MVC controllers", is a built-in security rule that is off by default [9]. Its implementation reports at the end of compilation, and only under this condition [10]:

```
        if (usingValidateAntiForgeryAttribute && !hasGlobalAntiForgeryFilter && ...)
```

The flag `usingValidateAntiForgeryAttribute` is set only when some controller or action in the project carries an attribute whose name matches `*Validate*Antiforgery*Attribute`. In a project that never uses such an attribute, the rule reports nothing.

CodeQL's query `cs/web/missing-token-validation` (severity error, precision high, security severity 8.8) applies the same condition and states the reason in its source [11]:

```
// Verify that validate anti forgery token attributes are used somewhere within this project, to
// avoid reporting false positives on projects that use an alternative approach to mitigate CSRF
// issues.
exists(getAValidatedElement())
```

A minimal reproduction confirms the behavior of CA5391 (Table 4). The same unprotected action is reported or not depending only on whether another action in the project is protected.

Table 4. CA5391 on an unprotected `[HttpPost]` action, .NET SDK 10.0.114, `AnalysisLevel=latest-all`.

The controller also contains	CA5391 on the unprotected action
A second action with `[ValidateAntiForgeryToken]`	reported
Nothing else	not reported

The trade-off is understandable: a project that mitigates CSRF another way, for example with token-based authentication, would otherwise receive a warning on every action. The consequence, however, inverts the risk. A project that protects most actions and forgets one is warned. A project with no antiforgery protection at all, which is the more dangerous case, is not. Neither rule's documentation states the condition. The CodeQL help text only hints at it, opening with "Web applications that use tokens to prevent cross-site request forgery (CSRF) should validate the tokens for all Http POST requests".

7.2 Semgrep: attribute syntax and global filters

Semgrep's rule `mvc-missing-antiforgery` (confidence LOW, subcategory audit) matches an action written as follows [12]:

```
        - pattern: |
            [$HTTPMETHOD]
            public IActionResult $METHOD(...){
                ...
            }
        - pattern-not: |
            [ValidateAntiForgeryToken]
            public IActionResult $METHOD(...){
                ...
            }
```

with `$HTTPMETHOD` restricted to `HttpPost`, `HttpPut`, `HttpDelete` and `HttpPatch`. Two problems follow (Table 5). First, the pattern `[$HTTPMETHOD]` does not match a verb attribute with a route template, such as `[HttpPost("email")]` in R03, so the unprotected action is missed. Second, the only accepted protection is `[ValidateAntiForgeryToken]` on the method itself, so a controller protected by the global `AutoValidateAntiforgeryToken` filter, the fix the ASP.NET Core documentation recommends, is reported as unprotected.

Table 5. Semgrep 1.183.0, semgrep-rules commit 16b9e6d, rule `mvc-missing-antiforgery`.

Action	Protection	Reported	Correct
`[HttpPost]`	none	yes	yes
`[HttpPost("email")]`	none	no	no (missed)
`[HttpPost]`	global `AutoValidateAntiforgeryToken` filter	yes	no (false positive)

With a route-less attribute, Semgrep would report both R03 variants alike and still could not tell the vulnerable application from the fixed one.

8 Discussion

For developers. A clean result from these tools says nothing about the four misconfigurations studied here. Today the only safeguard is the documentation, and a developer must read the right page to find it. A cheap complement is a test at the HTTP level, like the runner in this benchmark: send the request the protection should stop and assert that it is stopped. Such a test catches the misconfiguration regardless of how the pipeline is written.

For tool authors. Three of the four misconfigurations are ordering or combination errors in `Program.cs`, a single file with a predictable structure. Checks such as "`UseRateLimiter()` before `UseRouting()` while an endpoint uses `RequireRateLimiting()`" or "form-field `UseHttpMethodOverride` before `UseAntiforgery()`" are syntactic and could be added with low false-positive risk. For the MVC antiforgery rules, the condition "an antiforgery attribute is used somewhere" could be replaced or complemented by signals that a project serves forms with cookies, such as `AddControllersWithViews()`, Razor views or cookie authentication. At a minimum, the condition belongs in the rule's documentation, so that a silent result is not mistaken for a clean one. Semgrep's rule needs a pattern that accepts attribute arguments and recognition of the global filter.

For the framework. ASP.NET Core already refuses `AllowAnyOrigin()` together with `AllowCredentials()` at startup. The same reasoning could extend to the predicate form in R07 through an analyzer, and the pipeline could log a warning when an endpoint carries a rate limiting policy that the limiter cannot see. Such checks would turn silent misconfigurations into visible ones.

9 Threats to validity

Rule selection. The benchmark has four rules, chosen by the author and partly informed by his own contributions to ASP.NET Core. The results show that the tools miss these cases. They do not estimate how common the misconfigurations are in real applications.

Minimal applications. Real applications are larger, and tools may behave differently on them. The clearest example is the condition in CA5391 and CodeQL: in a project that uses an antiforgery attribute elsewhere, both would report the R03 action. Section 7 documents this dependence rather than hiding it.

Scoring. A detection counts only if its rule identifier appears on the vulnerable variant and not on the fixed one. A tool that reported the misconfiguration with the same identifier on both variants would be scored as not detecting it. All findings were inspected manually, and no such case occurred.

Tool coverage. Only three freely available tools were evaluated, each in its broadest documented configuration but without custom rules. Commercial analyzers and AI-based code review were not evaluated. Statements that a tool has no rule for a misconfiguration rest on listing its rule files and query names for the versions in Table 2.

Exploitability. The applications have no authentication, to stay minimal. In a real application, a forged request must carry the victim's cookie. ASP.NET Core authentication cookies default to `SameSite=Lax`, which modern browsers do not send on cross-site POST requests, so R01 and R03 are mainly exploitable when cookies use `SameSite=None`, from a same-site origin such as a sibling subdomain, or with clients that do not enforce SameSite. R07 was judged from the CORS response headers rather than in a browser, since the browser enforces CORS based on exactly those headers. R02 was measured with 10 requests against a limit of 3.

Environment. All runs used Linux and the .NET 8 and .NET 10 runtimes listed in Section 3. Earlier .NET versions were not tested.

10 Related work

Likaj, Khodayari and Pellegrino analyzed the CSRF defenses of 44 web frameworks, ASP.NET Core 3.1 among them [1]. Their study examines the frameworks' own mechanisms and defaults, not how applications configure them. This benchmark starts where that work stops: the framework's defense is sound, and the question is whether an application's configuration actually activates it.

Islam et al. studied the security configuration of Spring applications and described recurring insecure practices, based on a manual review of 28 projects [2]. Their work is the closest in spirit, an application-level view of a framework's security configuration, but it targets Java and Spring and does not evaluate detection tools.

Chen et al. measured CORS misconfigurations across live websites [3]. They observe the problem from outside, at the HTTP level, without regard to the framework that produced it. R07 shows one way such a configuration arises in ASP.NET Core and that current tools do not flag it.

The OWASP Benchmark is a long-standing test suite for measuring static analysis tools on Java code with known vulnerabilities [5]. This benchmark follows the same idea of paired true and false test cases, but for framework configuration in ASP.NET Core, with every case verified at runtime.

BaxBench measures whether large language models generate correct and secure backends, with 392 tasks across 14 frameworks in six languages, none of them C# or ASP.NET Core [4]. Its exploit-based checks are similar to the runtime verification used here.

11 Conclusion and future work

Four ASP.NET Core misconfigurations each disable a protection at runtime on .NET 8 and .NET 10 without any warning, and three widely used static analysis tools detect none of them. For the one case all three tools target, two stay silent by design in projects with no antiforgery protection at all, and the third does not match common attribute syntax. Until tools catch up, HTTP-level tests and documentation are the only safeguards against these cases.

Planned next steps:

- **More rules:** minimal API endpoints with `DisableAntiforgery()` under cookie authentication, `ShortCircuit()` on endpoints with authorization metadata, `[AllowAnonymous]` overriding `[Authorize]`, JWT validation switched off, weak authentication cookie settings, the developer exception page outside Development, and forwarded headers that trust any proxy.
- **More tools:** SonarAnalyzer for C#, Security Code Scan, and AI-based code review.
- **Reporting:** sharing the findings of Section 7 with the maintainers of the three tools.

Data availability

This report and everything it describes, namely the benchmark, the runner, the scan scripts, the reproductions of Section 7 and the continuous integration workflow, are published together in one repository [13] under the MIT license. They are archived on Zenodo as one record with the DOI 10.5281/zenodo.23822216, which covers all versions. This report describes version 1.0.0. The results were first produced with the code of version 0.1.0, and every push to the repository reruns the runtime checks and all three tools.

Use of AI

Parts of the text of this report were drafted with the assistance of a generative AI language model. The author reviewed and edited the text and is responsible for its content.

References

1. X. Likaj, S. Khodayari and G. Pellegrino. "Where We Stand (or Fall): An Analysis of CSRF Defenses in Web Frameworks." RAID 2021. doi.org/10.1145/3471621.3471846
2. M. Islam et al. "Coding Practices and Recommendations of Spring Security for Enterprise Applications." IEEE SecDev 2020. arXiv:2007.14319
3. J. Chen et al. "We Still Don't Have Secure Cross-Domain Requests: an Empirical Study of CORS." USENIX Security 2018. usenix.org
4. M. Vero et al. "BaxBench: Can LLMs Generate Correct and Secure Backends?" ICML 2025. arXiv:2502.11844
5. OWASP Foundation. "OWASP Benchmark Project." owasp.org
6. Microsoft. "Prevent Cross-Site Request Forgery (XSRF/CSRF) attacks in ASP.NET Core." learn.microsoft.com
7. Microsoft. "Rate limiting middleware in ASP.NET Core." learn.microsoft.com
8. Microsoft. "Enable Cross-Origin Requests (CORS) in ASP.NET Core." learn.microsoft.com
9. Microsoft. "CA5391: Use antiforgery tokens in ASP.NET Core MVC controllers." learn.microsoft.com
10. dotnet/roslyn-analyzers. `UseAutoValidateAntiforgeryToken.cs`, the implementation of CA5391. github.com
11. GitHub. CodeQL query `cs/web/missing-token-validation`, `MissingAntiForgeryTokenValidation.ql`, as shipped in csharp-queries 1.10.2. github.com
12. Semgrep. Rule `mvc-missing-antiforgery`, semgrep-rules commit 16b9e6d. github.com
13. G. Totinov. aspnetcore-security-bench: benchmark, runner, scan scripts and results for this report. GitHub, 2026. github.com/GrantTotinov/aspnetcore-security-bench
14. dotnet/aspnetcore, pull request #66772. github.com
15. dotnet/AspNetCore.Docs, pull request #37224. github.com
16. Microsoft. "ASP.NET Core middleware," section on middleware order. learn.microsoft.com
17. Microsoft. "HttpMethodOverrideExtensions.UseHttpMethodOverride Method." learn.microsoft.com

Web pages were accessed on 30 and 31 October 2026.